

DEEP LEARNING FOR FINANCE

MODULE 6 | LESSON 4

PORTFOLIO CHOICE WITH A SPARSE AUTOENCODER

Reading Time	90 minutes
---------------------	------------

Prior Knowledge	Neural Networks, machine learning, PCA, linear algebra
------------------------	--

Keywords	Autoencoder, portfolio choice
-----------------	-------------------------------

1. INTRODUCTION

In this notebook, we will build and train autoencoder models using the same data that we exploited in Lesson 2's notebook and some of the tools that we discussed in Lesson 3.

Additionally, we will implement the predictions of the autoencoder model to design a portfolio of stocks that maximizes the expected risk-weighted return in sample and compare its performance with other, more naïve strategies.

First of all, we load the libraries and data that we need for this notebook.

```
In [1]: import numpy as np
import pandas as pd
```

```
In [2]: import tensorflow as tf
```

```
2025-05-26 12:36:29.001036: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-05-26 12:36:29.052234: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2025-05-26 12:36:30.581926: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
```

```
In [3]: # List of tickers (same order)
tickers = [
    "AAPL", "MSFT", "GOOG", "AMZN", "TSLA",
    "TCEHY", "NVDA", "TSM", "META", "BABA",
```

```

    "ASML", "AVGO", "ORCL", "CSCO", "ADBE",
    "CRM", "NFLX", "TXN", "QCOM", "SAP"
]

# Read CSV directly with Date as index
print("Loading data from CSV...")
data = pd.read_csv('adjusted_prices.csv', index_col='Date', parse_dates=True)

print(f"Data loaded: {data.shape}")
print(f"Complete range: {data.index[0].date()} to {data.index[-1].date()}")

# Calculate daily returns (pct_change) for all tickers
print("Calculating daily returns...")
returns_data = data[tickers].pct_change()

# Remove first row (NaN from pct_change)
df = returns_data.dropna()

df.head()

```

Loading data from CSV...
 Data loaded: (1509, 20)
 Complete range: 2019-01-02 to 2024-12-30
 Calculating daily returns...

Out[3]:

	AAPL	MSFT	GOOG	AMZN	TSLA	TCEHY	NVDA	
Date								
2019-01-03 00:00:05:00	-0.099607	-0.036788	-0.028484	-0.025241	-0.031472	-0.028347	-0.060417	-C
2019-01-04 00:00:05:00	0.042689	0.046509	0.053786	0.050064	0.057697	0.058088	0.064068	C
2019-01-07 00:00:05:00	-0.002226	0.001275	-0.002167	0.034353	0.054361	0.006893	0.052940	C
2019-01-08 00:00:05:00	0.019063	0.007251	0.007385	0.016612	0.001164	-0.003423	-0.024895	-C
2019-01-09 00:00:05:00	0.016981	0.014300	-0.001505	0.001714	0.009483	0.038518	0.019667	C

In [4]:

```

print(f"Final DataFrame: {df.shape}")
print("Tickers included:", df.columns.tolist())

# Train/test split (80/20)
train_size = int(0.8 * df.shape[0])

```

```
Y_train = df.iloc[:train_size + 1].to_numpy()
Y_test = df.iloc[train_size + 1:].to_numpy()
```

Final DataFrame: (1508, 20)

Tickers included: ['AAPL', 'MSFT', 'GOOG', 'AMZN', 'TSLA', 'TCEHY', 'NVDA', 'TSM', 'META', 'BABA', 'ASML', 'AVGO', 'ORCL', 'CSCO', 'ADBE', 'CRM', 'NFLX', 'TXN', 'QCOM', 'SAP']

```
In [5]: print(f"\nBeginning of training sample: {df.index[0].date()}")
print(f"End of training sample: {df.index[train_size].date()}")
print(f"End of test sample: {df.index[-1].date()}")
print(f"Training samples: {Y_train.shape}")
print(f"Test samples: {Y_test.shape}")
```

Beginning of training sample: 2019-01-03

End of training sample: 2023-10-18

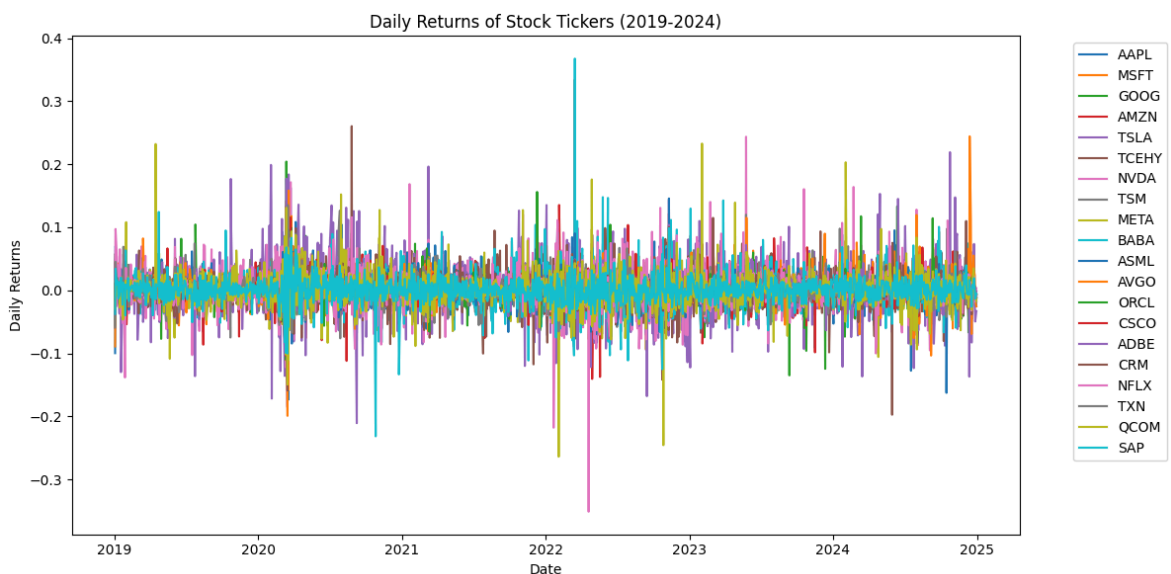
End of test sample: 2024-12-30

Training samples: (1207, 20)

Test samples: (301, 20)

```
In [6]: import matplotlib.pyplot as plt

plt.figure(figsize=(12, 6))
plt.plot(df.index, df)
plt.title(f'Daily Returns of Stock Tickers ({df.index[0].year}-{df.index[-1].year})')
plt.xlabel('Date')
plt.ylabel('Daily Returns')
plt.legend(df.columns, bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



2. BUILDING AND TRAINING A SPARSE AUTOENCODER

We will design a portfolio based on the predictions of an autoencoder model with a loss function that includes a regularization term on the value of the factors generated by the model.

In TensorFlow, we add an L1-regularizer in the bottleneck

"tf.keras.regularizers.l1(l1weight)", where "l1weight" is a hyperparameter that governs the

degree of regularization on the factor space. This parameter will determine the actual dimension of the bottleneck, without us having to impose it beforehand.

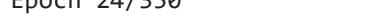
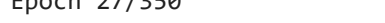
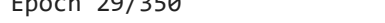
```
In [7]: class sparse_autoencoder(tf.keras.Model):
    def __init__(self, output_dim, bottleneck_dim=1, l1weight=0.0):
        super(sparse_autoencoder, self).__init__()
        self.output_dim = output_dim
        self.bottleneck_dim = bottleneck_dim
        self.l1weight = l1weight
        initializer = tf.keras.initializers.GlorotUniform()
        # Set the activity regularizer that will generate the sparsity of factor
        activity_regularizer = tf.keras.regularizers.l1(l1weight)
        # Build the encoding layer, including the activity regularizer
        self.encoder = tf.keras.Sequential(
            [
                tf.keras.layers.Dense(
                    bottleneck_dim,
                    activity_regularizer=activity_regularizer,
                    kernel_initializer=initializer,
                    bias_initializer=tf.keras.initializers.Zeros(),
                )
            ]
        )
        # Build the decoding layer
        self.decoder = tf.keras.Sequential(
            [
                tf.keras.layers.Dense(
                    output_dim,
                    kernel_initializer=initializer,
                    bias_initializer=tf.keras.initializers.Zeros(),
                )
            ]
        )

    def call(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded
```

Let's compile and train the model. We set the L1-regularization parameter to $\lambda_1=0.0025$.

```
In [8]: # Set the bottleneck size equal to the input size
nlatent = Y_train.shape[1]
# Set the parameter on the L1 activity-regularization term
l1weight = 0.0025
# Choose the proportion of training data used for validation
val_split = 0.2
# Determine early stopping criterion
early_stop = tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=50)
# Choose optimizer
adam = tf.keras.optimizers.Adam(learning_rate=1e-4)
# Build and compile the model
model = sparse_autoencoder(Y_train.shape[1], nlatent, l1weight)
model.compile(optimizer=adam, loss="mse")
# Train the model
model.fit(
    x=Y_train,
```

```
y=Y_train,  
batch_size=32,  
epochs=350,  
validation_split=val_split,  
callbacks=[early_stop],  
verbose=1,  
)
```

```
Epoch 1/350
31/31  1s 8ms/step - loss: 0.0273 - val_loss: 0.0245
Epoch 2/350
31/31  0s 4ms/step - loss: 0.0267 - val_loss: 0.0239
Epoch 3/350
31/31  0s 4ms/step - loss: 0.0268 - val_loss: 0.0234
Epoch 4/350
31/31  0s 5ms/step - loss: 0.0252 - val_loss: 0.0229
Epoch 5/350
31/31  0s 4ms/step - loss: 0.0241 - val_loss: 0.0225
Epoch 6/350
31/31  0s 4ms/step - loss: 0.0243 - val_loss: 0.0220
Epoch 7/350
31/31  0s 4ms/step - loss: 0.0238 - val_loss: 0.0216
Epoch 8/350
31/31  0s 4ms/step - loss: 0.0231 - val_loss: 0.0213
Epoch 9/350
31/31  0s 4ms/step - loss: 0.0232 - val_loss: 0.0209
Epoch 10/350
31/31  0s 4ms/step - loss: 0.0224 - val_loss: 0.0206
Epoch 11/350
31/31  0s 3ms/step - loss: 0.0214 - val_loss: 0.0203
Epoch 12/350
31/31  0s 4ms/step - loss: 0.0210 - val_loss: 0.0200
Epoch 13/350
31/31  0s 4ms/step - loss: 0.0214 - val_loss: 0.0197
Epoch 14/350
31/31  0s 4ms/step - loss: 0.0216 - val_loss: 0.0195
Epoch 15/350
31/31  0s 4ms/step - loss: 0.0200 - val_loss: 0.0192
Epoch 16/350
31/31  0s 4ms/step - loss: 0.0202 - val_loss: 0.0190
Epoch 17/350
31/31  0s 3ms/step - loss: 0.0204 - val_loss: 0.0188
Epoch 18/350
31/31  0s 4ms/step - loss: 0.0198 - val_loss: 0.0186
Epoch 19/350
31/31  0s 4ms/step - loss: 0.0196 - val_loss: 0.0184
Epoch 20/350
31/31  0s 4ms/step - loss: 0.0188 - val_loss: 0.0182
Epoch 21/350
31/31  0s 4ms/step - loss: 0.0183 - val_loss: 0.0180
Epoch 22/350
31/31  0s 4ms/step - loss: 0.0185 - val_loss: 0.0179
Epoch 23/350
31/31  0s 4ms/step - loss: 0.0186 - val_loss: 0.0177
Epoch 24/350
31/31  0s 4ms/step - loss: 0.0184 - val_loss: 0.0175
Epoch 25/350
31/31  0s 3ms/step - loss: 0.0181 - val_loss: 0.0174
Epoch 26/350
31/31  0s 4ms/step - loss: 0.0177 - val_loss: 0.0172
Epoch 27/350
31/31  0s 4ms/step - loss: 0.0181 - val_loss: 0.0171
Epoch 28/350
31/31  0s 4ms/step - loss: 0.0176 - val_loss: 0.0169
Epoch 29/350
31/31  0s 4ms/step - loss: 0.0171 - val_loss: 0.0168
Epoch 30/350
31/31  0s 3ms/step - loss: 0.0170 - val_loss: 0.0166
```

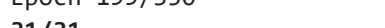
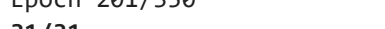
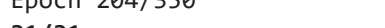
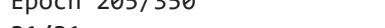
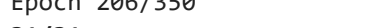
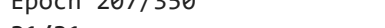
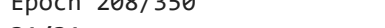
```
Epoch 31/350
31/31  0s 3ms/step - loss: 0.0167 - val_loss: 0.0165
Epoch 32/350
31/31  0s 3ms/step - loss: 0.0171 - val_loss: 0.0163
Epoch 33/350
31/31  0s 4ms/step - loss: 0.0171 - val_loss: 0.0162
Epoch 34/350
31/31  0s 4ms/step - loss: 0.0164 - val_loss: 0.0161
Epoch 35/350
31/31  0s 4ms/step - loss: 0.0165 - val_loss: 0.0159
Epoch 36/350
31/31  0s 4ms/step - loss: 0.0160 - val_loss: 0.0158
Epoch 37/350
31/31  0s 4ms/step - loss: 0.0165 - val_loss: 0.0156
Epoch 38/350
31/31  0s 4ms/step - loss: 0.0163 - val_loss: 0.0155
Epoch 39/350
31/31  0s 3ms/step - loss: 0.0161 - val_loss: 0.0154
Epoch 40/350
31/31  0s 4ms/step - loss: 0.0154 - val_loss: 0.0152
Epoch 41/350
31/31  0s 4ms/step - loss: 0.0158 - val_loss: 0.0151
Epoch 42/350
31/31  0s 4ms/step - loss: 0.0150 - val_loss: 0.0150
Epoch 43/350
31/31  0s 3ms/step - loss: 0.0156 - val_loss: 0.0148
Epoch 44/350
31/31  0s 4ms/step - loss: 0.0151 - val_loss: 0.0147
Epoch 45/350
31/31  0s 4ms/step - loss: 0.0148 - val_loss: 0.0146
Epoch 46/350
31/31  0s 4ms/step - loss: 0.0147 - val_loss: 0.0144
Epoch 47/350
31/31  0s 5ms/step - loss: 0.0147 - val_loss: 0.0143
Epoch 48/350
31/31  0s 4ms/step - loss: 0.0147 - val_loss: 0.0142
Epoch 49/350
31/31  0s 4ms/step - loss: 0.0144 - val_loss: 0.0140
Epoch 50/350
31/31  0s 4ms/step - loss: 0.0146 - val_loss: 0.0139
Epoch 51/350
31/31  0s 3ms/step - loss: 0.0141 - val_loss: 0.0138
Epoch 52/350
31/31  0s 3ms/step - loss: 0.0140 - val_loss: 0.0136
Epoch 53/350
31/31  0s 4ms/step - loss: 0.0139 - val_loss: 0.0135
Epoch 54/350
31/31  0s 3ms/step - loss: 0.0134 - val_loss: 0.0134
Epoch 55/350
31/31  0s 4ms/step - loss: 0.0136 - val_loss: 0.0133
Epoch 56/350
31/31  0s 4ms/step - loss: 0.0135 - val_loss: 0.0131
Epoch 57/350
31/31  0s 4ms/step - loss: 0.0133 - val_loss: 0.0130
Epoch 58/350
31/31  0s 4ms/step - loss: 0.0132 - val_loss: 0.0129
Epoch 59/350
31/31  0s 4ms/step - loss: 0.0130 - val_loss: 0.0127
Epoch 60/350
31/31  0s 4ms/step - loss: 0.0128 - val_loss: 0.0126
```

```
Epoch 61/350
31/31  0s 4ms/step - loss: 0.0126 - val_loss: 0.0125
Epoch 62/350
31/31  0s 4ms/step - loss: 0.0127 - val_loss: 0.0124
Epoch 63/350
31/31  0s 4ms/step - loss: 0.0123 - val_loss: 0.0122
Epoch 64/350
31/31  0s 3ms/step - loss: 0.0124 - val_loss: 0.0121
Epoch 65/350
31/31  0s 4ms/step - loss: 0.0121 - val_loss: 0.0120
Epoch 66/350
31/31  0s 4ms/step - loss: 0.0122 - val_loss: 0.0119
Epoch 67/350
31/31  0s 4ms/step - loss: 0.0117 - val_loss: 0.0118
Epoch 68/350
31/31  0s 4ms/step - loss: 0.0118 - val_loss: 0.0116
Epoch 69/350
31/31  0s 3ms/step - loss: 0.0118 - val_loss: 0.0115
Epoch 70/350
31/31  0s 4ms/step - loss: 0.0118 - val_loss: 0.0114
Epoch 71/350
31/31  0s 4ms/step - loss: 0.0114 - val_loss: 0.0113
Epoch 72/350
31/31  0s 4ms/step - loss: 0.0114 - val_loss: 0.0112
Epoch 73/350
31/31  0s 4ms/step - loss: 0.0113 - val_loss: 0.0110
Epoch 74/350
31/31  0s 4ms/step - loss: 0.0109 - val_loss: 0.0109
Epoch 75/350
31/31  0s 4ms/step - loss: 0.0106 - val_loss: 0.0108
Epoch 76/350
31/31  0s 4ms/step - loss: 0.0106 - val_loss: 0.0107
Epoch 77/350
31/31  0s 3ms/step - loss: 0.0106 - val_loss: 0.0106
Epoch 78/350
31/31  0s 3ms/step - loss: 0.0107 - val_loss: 0.0104
Epoch 79/350
31/31  0s 4ms/step - loss: 0.0103 - val_loss: 0.0103
Epoch 80/350
31/31  0s 4ms/step - loss: 0.0104 - val_loss: 0.0102
Epoch 81/350
31/31  0s 4ms/step - loss: 0.0100 - val_loss: 0.0101
Epoch 82/350
31/31  0s 3ms/step - loss: 0.0102 - val_loss: 0.0100
Epoch 83/350
31/31  0s 4ms/step - loss: 0.0099 - val_loss: 0.0099
Epoch 84/350
31/31  0s 4ms/step - loss: 0.0099 - val_loss: 0.0097
Epoch 85/350
31/31  0s 4ms/step - loss: 0.0097 - val_loss: 0.0096
Epoch 86/350
31/31  0s 4ms/step - loss: 0.0096 - val_loss: 0.0095
Epoch 87/350
31/31  0s 4ms/step - loss: 0.0094 - val_loss: 0.0094
Epoch 88/350
31/31  0s 4ms/step - loss: 0.0094 - val_loss: 0.0093
Epoch 89/350
31/31  0s 4ms/step - loss: 0.0094 - val_loss: 0.0092
Epoch 90/350
31/31  0s 4ms/step - loss: 0.0091 - val_loss: 0.0091
```

```
Epoch 91/350
31/31  0s 3ms/step - loss: 0.0091 - val_loss: 0.0089
Epoch 92/350
31/31  0s 4ms/step - loss: 0.0090 - val_loss: 0.0088
Epoch 93/350
31/31  0s 4ms/step - loss: 0.0084 - val_loss: 0.0087
Epoch 94/350
31/31  0s 4ms/step - loss: 0.0087 - val_loss: 0.0086
Epoch 95/350
31/31  0s 4ms/step - loss: 0.0086 - val_loss: 0.0085
Epoch 96/350
31/31  0s 4ms/step - loss: 0.0084 - val_loss: 0.0084
Epoch 97/350
31/31  0s 3ms/step - loss: 0.0084 - val_loss: 0.0083
Epoch 98/350
31/31  0s 4ms/step - loss: 0.0082 - val_loss: 0.0082
Epoch 99/350
31/31  0s 4ms/step - loss: 0.0082 - val_loss: 0.0081
Epoch 100/350
31/31  0s 4ms/step - loss: 0.0078 - val_loss: 0.0080
Epoch 101/350
31/31  0s 3ms/step - loss: 0.0078 - val_loss: 0.0079
Epoch 102/350
31/31  0s 3ms/step - loss: 0.0079 - val_loss: 0.0078
Epoch 103/350
31/31  0s 4ms/step - loss: 0.0078 - val_loss: 0.0077
Epoch 104/350
31/31  0s 4ms/step - loss: 0.0075 - val_loss: 0.0076
Epoch 105/350
31/31  0s 4ms/step - loss: 0.0074 - val_loss: 0.0075
Epoch 106/350
31/31  0s 3ms/step - loss: 0.0075 - val_loss: 0.0074
Epoch 107/350
31/31  0s 4ms/step - loss: 0.0071 - val_loss: 0.0073
Epoch 108/350
31/31  0s 4ms/step - loss: 0.0072 - val_loss: 0.0072
Epoch 109/350
31/31  0s 4ms/step - loss: 0.0073 - val_loss: 0.0071
Epoch 110/350
31/31  0s 6ms/step - loss: 0.0069 - val_loss: 0.0070
Epoch 111/350
31/31  0s 5ms/step - loss: 0.0071 - val_loss: 0.0069
Epoch 112/350
31/31  0s 4ms/step - loss: 0.0067 - val_loss: 0.0068
Epoch 113/350
31/31  0s 4ms/step - loss: 0.0066 - val_loss: 0.0067
Epoch 114/350
31/31  0s 4ms/step - loss: 0.0067 - val_loss: 0.0066
Epoch 115/350
31/31  0s 4ms/step - loss: 0.0064 - val_loss: 0.0065
Epoch 116/350
31/31  0s 4ms/step - loss: 0.0065 - val_loss: 0.0064
Epoch 117/350
31/31  0s 4ms/step - loss: 0.0063 - val_loss: 0.0063
Epoch 118/350
31/31  0s 5ms/step - loss: 0.0063 - val_loss: 0.0062
Epoch 119/350
31/31  0s 4ms/step - loss: 0.0060 - val_loss: 0.0061
Epoch 120/350
31/31  0s 4ms/step - loss: 0.0061 - val_loss: 0.0060
```

```
Epoch 121/350
31/31  0s 4ms/step - loss: 0.0059 - val_loss: 0.0060
Epoch 122/350
31/31  0s 4ms/step - loss: 0.0057 - val_loss: 0.0059
Epoch 123/350
31/31  0s 4ms/step - loss: 0.0056 - val_loss: 0.0058
Epoch 124/350
31/31  0s 3ms/step - loss: 0.0056 - val_loss: 0.0057
Epoch 125/350
31/31  0s 4ms/step - loss: 0.0057 - val_loss: 0.0056
Epoch 126/350
31/31  0s 4ms/step - loss: 0.0054 - val_loss: 0.0055
Epoch 127/350
31/31  0s 4ms/step - loss: 0.0055 - val_loss: 0.0054
Epoch 128/350
31/31  0s 3ms/step - loss: 0.0052 - val_loss: 0.0053
Epoch 129/350
31/31  0s 4ms/step - loss: 0.0053 - val_loss: 0.0052
Epoch 130/350
31/31  0s 4ms/step - loss: 0.0050 - val_loss: 0.0052
Epoch 131/350
31/31  0s 4ms/step - loss: 0.0050 - val_loss: 0.0051
Epoch 132/350
31/31  0s 4ms/step - loss: 0.0049 - val_loss: 0.0050
Epoch 133/350
31/31  0s 4ms/step - loss: 0.0050 - val_loss: 0.0049
Epoch 134/350
31/31  0s 4ms/step - loss: 0.0048 - val_loss: 0.0048
Epoch 135/350
31/31  0s 4ms/step - loss: 0.0046 - val_loss: 0.0048
Epoch 136/350
31/31  0s 4ms/step - loss: 0.0046 - val_loss: 0.0047
Epoch 137/350
31/31  0s 4ms/step - loss: 0.0045 - val_loss: 0.0046
Epoch 138/350
31/31  0s 3ms/step - loss: 0.0046 - val_loss: 0.0045
Epoch 139/350
31/31  0s 4ms/step - loss: 0.0044 - val_loss: 0.0044
Epoch 140/350
31/31  0s 3ms/step - loss: 0.0044 - val_loss: 0.0043
Epoch 141/350
31/31  0s 4ms/step - loss: 0.0042 - val_loss: 0.0043
Epoch 142/350
31/31  0s 4ms/step - loss: 0.0042 - val_loss: 0.0042
Epoch 143/350
31/31  0s 3ms/step - loss: 0.0040 - val_loss: 0.0041
Epoch 144/350
31/31  0s 4ms/step - loss: 0.0039 - val_loss: 0.0040
Epoch 145/350
31/31  0s 4ms/step - loss: 0.0041 - val_loss: 0.0039
Epoch 146/350
31/31  0s 4ms/step - loss: 0.0039 - val_loss: 0.0039
Epoch 147/350
31/31  0s 4ms/step - loss: 0.0037 - val_loss: 0.0038
Epoch 148/350
31/31  0s 4ms/step - loss: 0.0035 - val_loss: 0.0037
Epoch 149/350
31/31  0s 3ms/step - loss: 0.0036 - val_loss: 0.0037
Epoch 150/350
31/31  0s 4ms/step - loss: 0.0035 - val_loss: 0.0036
```

```
Epoch 151/350
31/31  0s 4ms/step - loss: 0.0035 - val_loss: 0.0035
Epoch 152/350
31/31  0s 4ms/step - loss: 0.0033 - val_loss: 0.0034
Epoch 153/350
31/31  0s 4ms/step - loss: 0.0034 - val_loss: 0.0034
Epoch 154/350
31/31  0s 4ms/step - loss: 0.0033 - val_loss: 0.0033
Epoch 155/350
31/31  0s 4ms/step - loss: 0.0032 - val_loss: 0.0032
Epoch 156/350
31/31  0s 3ms/step - loss: 0.0032 - val_loss: 0.0032
Epoch 157/350
31/31  0s 5ms/step - loss: 0.0030 - val_loss: 0.0031
Epoch 158/350
31/31  0s 4ms/step - loss: 0.0029 - val_loss: 0.0030
Epoch 159/350
31/31  0s 4ms/step - loss: 0.0029 - val_loss: 0.0030
Epoch 160/350
31/31  0s 3ms/step - loss: 0.0028 - val_loss: 0.0029
Epoch 161/350
31/31  0s 4ms/step - loss: 0.0029 - val_loss: 0.0028
Epoch 162/350
31/31  0s 4ms/step - loss: 0.0027 - val_loss: 0.0027
Epoch 163/350
31/31  0s 4ms/step - loss: 0.0027 - val_loss: 0.0027
Epoch 164/350
31/31  0s 4ms/step - loss: 0.0026 - val_loss: 0.0026
Epoch 165/350
31/31  0s 4ms/step - loss: 0.0025 - val_loss: 0.0025
Epoch 166/350
31/31  0s 4ms/step - loss: 0.0024 - val_loss: 0.0025
Epoch 167/350
31/31  0s 4ms/step - loss: 0.0024 - val_loss: 0.0024
Epoch 168/350
31/31  0s 3ms/step - loss: 0.0024 - val_loss: 0.0024
Epoch 169/350
31/31  0s 4ms/step - loss: 0.0024 - val_loss: 0.0023
Epoch 170/350
31/31  0s 3ms/step - loss: 0.0022 - val_loss: 0.0022
Epoch 171/350
31/31  0s 3ms/step - loss: 0.0022 - val_loss: 0.0022
Epoch 172/350
31/31  0s 4ms/step - loss: 0.0021 - val_loss: 0.0021
Epoch 173/350
31/31  0s 4ms/step - loss: 0.0021 - val_loss: 0.0021
Epoch 174/350
31/31  0s 3ms/step - loss: 0.0020 - val_loss: 0.0020
Epoch 175/350
31/31  0s 4ms/step - loss: 0.0020 - val_loss: 0.0020
Epoch 176/350
31/31  0s 4ms/step - loss: 0.0019 - val_loss: 0.0019
Epoch 177/350
31/31  0s 4ms/step - loss: 0.0019 - val_loss: 0.0019
Epoch 178/350
31/31  0s 4ms/step - loss: 0.0018 - val_loss: 0.0018
Epoch 179/350
31/31  0s 4ms/step - loss: 0.0018 - val_loss: 0.0018
Epoch 180/350
31/31  0s 4ms/step - loss: 0.0018 - val_loss: 0.0017
```

```
Epoch 181/350
31/31  0s 3ms/step - loss: 0.0017 - val_loss: 0.0017
Epoch 182/350
31/31  0s 4ms/step - loss: 0.0017 - val_loss: 0.0016
Epoch 183/350
31/31  0s 4ms/step - loss: 0.0017 - val_loss: 0.0016
Epoch 184/350
31/31  0s 4ms/step - loss: 0.0017 - val_loss: 0.0016
Epoch 185/350
31/31  0s 4ms/step - loss: 0.0016 - val_loss: 0.0015
Epoch 186/350
31/31  0s 3ms/step - loss: 0.0015 - val_loss: 0.0015
Epoch 187/350
31/31  0s 3ms/step - loss: 0.0015 - val_loss: 0.0014
Epoch 188/350
31/31  0s 4ms/step - loss: 0.0014 - val_loss: 0.0014
Epoch 189/350
31/31  0s 4ms/step - loss: 0.0014 - val_loss: 0.0014
Epoch 190/350
31/31  0s 4ms/step - loss: 0.0014 - val_loss: 0.0013
Epoch 191/350
31/31  0s 3ms/step - loss: 0.0013 - val_loss: 0.0013
Epoch 192/350
31/31  0s 4ms/step - loss: 0.0013 - val_loss: 0.0012
Epoch 193/350
31/31  0s 3ms/step - loss: 0.0012 - val_loss: 0.0012
Epoch 194/350
31/31  0s 4ms/step - loss: 0.0012 - val_loss: 0.0012
Epoch 195/350
31/31  0s 4ms/step - loss: 0.0012 - val_loss: 0.0011
Epoch 196/350
31/31  0s 4ms/step - loss: 0.0011 - val_loss: 0.0011
Epoch 197/350
31/31  0s 4ms/step - loss: 0.0011 - val_loss: 0.0011
Epoch 198/350
31/31  0s 4ms/step - loss: 0.0012 - val_loss: 0.0010
Epoch 199/350
31/31  0s 4ms/step - loss: 0.0011 - val_loss: 0.0010
Epoch 200/350
31/31  0s 4ms/step - loss: 0.0011 - val_loss: 9.9598e-04
Epoch 201/350
31/31  0s 4ms/step - loss: 0.0010 - val_loss: 9.6657e-04
Epoch 202/350
31/31  0s 4ms/step - loss: 0.0010 - val_loss: 9.4100e-04
Epoch 203/350
31/31  0s 4ms/step - loss: 0.0010 - val_loss: 9.2630e-04
Epoch 204/350
31/31  0s 4ms/step - loss: 9.4837e-04 - val_loss: 9.0591e-04
Epoch 205/350
31/31  0s 3ms/step - loss: 0.0010 - val_loss: 8.8512e-04
Epoch 206/350
31/31  0s 4ms/step - loss: 0.0010 - val_loss: 8.5939e-04
Epoch 207/350
31/31  0s 4ms/step - loss: 9.4026e-04 - val_loss: 8.5006e-04
Epoch 208/350
31/31  0s 4ms/step - loss: 8.9723e-04 - val_loss: 8.3320e-04
Epoch 209/350
31/31  0s 4ms/step - loss: 9.0880e-04 - val_loss: 8.0905e-04
Epoch 210/350
31/31  0s 4ms/step - loss: 8.8811e-04 - val_loss: 8.0341e-04
```

```
Epoch 211/350
31/31  0s 4ms/step - loss: 8.6462e-04 - val_loss: 7.8496e-04
Epoch 212/350
31/31  0s 4ms/step - loss: 9.6199e-04 - val_loss: 7.6977e-04
Epoch 213/350
31/31  0s 4ms/step - loss: 8.7442e-04 - val_loss: 7.5621e-04
Epoch 214/350
31/31  0s 4ms/step - loss: 7.9466e-04 - val_loss: 7.4551e-04
Epoch 215/350
31/31  0s 4ms/step - loss: 8.4721e-04 - val_loss: 7.3255e-04
Epoch 216/350
31/31  0s 4ms/step - loss: 7.9239e-04 - val_loss: 7.2596e-04
Epoch 217/350
31/31  0s 4ms/step - loss: 8.6110e-04 - val_loss: 7.1625e-04
Epoch 218/350
31/31  0s 4ms/step - loss: 7.5705e-04 - val_loss: 7.0012e-04
Epoch 219/350
31/31  0s 4ms/step - loss: 7.4169e-04 - val_loss: 6.9393e-04
Epoch 220/350
31/31  0s 4ms/step - loss: 7.7664e-04 - val_loss: 6.9053e-04
Epoch 221/350
31/31  0s 4ms/step - loss: 7.9245e-04 - val_loss: 6.7629e-04
Epoch 222/350
31/31  0s 4ms/step - loss: 7.8312e-04 - val_loss: 6.7416e-04
Epoch 223/350
31/31  0s 4ms/step - loss: 7.2096e-04 - val_loss: 6.6117e-04
Epoch 224/350
31/31  0s 4ms/step - loss: 7.1062e-04 - val_loss: 6.5456e-04
Epoch 225/350
31/31  0s 4ms/step - loss: 7.7285e-04 - val_loss: 6.4517e-04
Epoch 226/350
31/31  0s 4ms/step - loss: 7.5388e-04 - val_loss: 6.4444e-04
Epoch 227/350
31/31  0s 3ms/step - loss: 7.4369e-04 - val_loss: 6.3181e-04
Epoch 228/350
31/31  0s 3ms/step - loss: 6.8822e-04 - val_loss: 6.2895e-04
Epoch 229/350
31/31  0s 4ms/step - loss: 6.6178e-04 - val_loss: 6.2198e-04
Epoch 230/350
31/31  0s 3ms/step - loss: 7.3060e-04 - val_loss: 6.1930e-04
Epoch 231/350
31/31  0s 4ms/step - loss: 6.6227e-04 - val_loss: 6.1389e-04
Epoch 232/350
31/31  0s 3ms/step - loss: 7.1834e-04 - val_loss: 6.0468e-04
Epoch 233/350
31/31  0s 4ms/step - loss: 6.9119e-04 - val_loss: 6.0104e-04
Epoch 234/350
31/31  0s 4ms/step - loss: 7.3072e-04 - val_loss: 5.9892e-04
Epoch 235/350
31/31  0s 4ms/step - loss: 7.1064e-04 - val_loss: 5.9482e-04
Epoch 236/350
31/31  0s 4ms/step - loss: 6.8085e-04 - val_loss: 5.9023e-04
Epoch 237/350
31/31  0s 4ms/step - loss: 7.3317e-04 - val_loss: 5.8188e-04
Epoch 238/350
31/31  0s 4ms/step - loss: 7.2154e-04 - val_loss: 5.8049e-04
Epoch 239/350
31/31  0s 4ms/step - loss: 7.2158e-04 - val_loss: 5.8148e-04
Epoch 240/350
31/31  0s 4ms/step - loss: 7.0546e-04 - val_loss: 5.7321e-04
```

```
Epoch 241/350
31/31  0s 4ms/step - loss: 7.0606e-04 - val_loss: 5.8069e-04
Epoch 242/350
31/31  0s 3ms/step - loss: 6.5722e-04 - val_loss: 5.7367e-04
Epoch 243/350
31/31  0s 4ms/step - loss: 6.5207e-04 - val_loss: 5.7832e-04
Epoch 244/350
31/31  0s 4ms/step - loss: 6.5185e-04 - val_loss: 5.7417e-04
Epoch 245/350
31/31  0s 4ms/step - loss: 6.8971e-04 - val_loss: 5.7500e-04
Epoch 246/350
31/31  0s 4ms/step - loss: 6.9838e-04 - val_loss: 5.7275e-04
Epoch 247/350
31/31  0s 4ms/step - loss: 6.6729e-04 - val_loss: 5.7409e-04
Epoch 248/350
31/31  0s 4ms/step - loss: 6.7206e-04 - val_loss: 5.7674e-04
Epoch 249/350
31/31  0s 3ms/step - loss: 7.1314e-04 - val_loss: 5.7268e-04
Epoch 250/350
31/31  0s 4ms/step - loss: 6.7272e-04 - val_loss: 5.7438e-04
Epoch 251/350
31/31  0s 4ms/step - loss: 6.4517e-04 - val_loss: 5.7017e-04
Epoch 252/350
31/31  0s 4ms/step - loss: 6.8427e-04 - val_loss: 5.7634e-04
Epoch 253/350
31/31  0s 4ms/step - loss: 7.2278e-04 - val_loss: 5.7780e-04
Epoch 254/350
31/31  0s 4ms/step - loss: 6.2318e-04 - val_loss: 5.7251e-04
Epoch 255/350
31/31  0s 5ms/step - loss: 7.1412e-04 - val_loss: 5.7501e-04
Epoch 256/350
31/31  0s 4ms/step - loss: 7.0545e-04 - val_loss: 5.7014e-04
Epoch 257/350
31/31  0s 4ms/step - loss: 6.5516e-04 - val_loss: 5.7730e-04
Epoch 258/350
31/31  0s 4ms/step - loss: 6.9275e-04 - val_loss: 5.7832e-04
Epoch 259/350
31/31  0s 4ms/step - loss: 6.2398e-04 - val_loss: 5.7191e-04
Epoch 260/350
31/31  0s 4ms/step - loss: 6.8853e-04 - val_loss: 5.6961e-04
Epoch 261/350
31/31  0s 3ms/step - loss: 7.0878e-04 - val_loss: 5.7236e-04
Epoch 262/350
31/31  0s 3ms/step - loss: 7.1000e-04 - val_loss: 5.7069e-04
Epoch 263/350
31/31  0s 4ms/step - loss: 6.3854e-04 - val_loss: 5.7092e-04
Epoch 264/350
31/31  0s 3ms/step - loss: 6.2705e-04 - val_loss: 5.6668e-04
Epoch 265/350
31/31  0s 3ms/step - loss: 6.5795e-04 - val_loss: 5.7109e-04
Epoch 266/350
31/31  0s 4ms/step - loss: 6.3558e-04 - val_loss: 5.7240e-04
Epoch 267/350
31/31  0s 4ms/step - loss: 6.6518e-04 - val_loss: 5.7040e-04
Epoch 268/350
31/31  0s 4ms/step - loss: 6.9295e-04 - val_loss: 5.6849e-04
Epoch 269/350
31/31  0s 3ms/step - loss: 6.7890e-04 - val_loss: 5.7241e-04
Epoch 270/350
31/31  0s 4ms/step - loss: 6.5882e-04 - val_loss: 5.6973e-04
```

```
Epoch 271/350
31/31  0s 4ms/step - loss: 6.8025e-04 - val_loss: 5.7501e-04
Epoch 272/350
31/31  0s 4ms/step - loss: 6.9398e-04 - val_loss: 5.7323e-04
Epoch 273/350
31/31  0s 3ms/step - loss: 6.7430e-04 - val_loss: 5.7056e-04
Epoch 274/350
31/31  0s 4ms/step - loss: 6.5795e-04 - val_loss: 5.6806e-04
Epoch 275/350
31/31  0s 4ms/step - loss: 6.8659e-04 - val_loss: 5.7478e-04
Epoch 276/350
31/31  0s 4ms/step - loss: 6.4343e-04 - val_loss: 5.6935e-04
Epoch 277/350
31/31  0s 4ms/step - loss: 6.3373e-04 - val_loss: 5.7445e-04
Epoch 278/350
31/31  0s 4ms/step - loss: 7.3480e-04 - val_loss: 5.7532e-04
Epoch 279/350
31/31  0s 4ms/step - loss: 7.5003e-04 - val_loss: 5.7371e-04
Epoch 280/350
31/31  0s 4ms/step - loss: 7.1944e-04 - val_loss: 5.7226e-04
Epoch 281/350
31/31  0s 4ms/step - loss: 6.6910e-04 - val_loss: 5.7201e-04
Epoch 282/350
31/31  0s 4ms/step - loss: 6.4400e-04 - val_loss: 5.7376e-04
Epoch 283/350
31/31  0s 3ms/step - loss: 6.4126e-04 - val_loss: 5.7532e-04
Epoch 284/350
31/31  0s 4ms/step - loss: 6.5850e-04 - val_loss: 5.7647e-04
Epoch 285/350
31/31  0s 4ms/step - loss: 7.3727e-04 - val_loss: 5.7140e-04
Epoch 286/350
31/31  0s 3ms/step - loss: 7.0063e-04 - val_loss: 5.7145e-04
Epoch 287/350
31/31  0s 4ms/step - loss: 6.8961e-04 - val_loss: 5.7049e-04
Epoch 288/350
31/31  0s 4ms/step - loss: 6.5630e-04 - val_loss: 5.7349e-04
Epoch 289/350
31/31  0s 4ms/step - loss: 6.7309e-04 - val_loss: 5.7201e-04
Epoch 290/350
31/31  0s 3ms/step - loss: 6.4513e-04 - val_loss: 5.7040e-04
Epoch 291/350
31/31  0s 3ms/step - loss: 6.8965e-04 - val_loss: 5.7257e-04
Epoch 292/350
31/31  0s 4ms/step - loss: 7.5192e-04 - val_loss: 5.7336e-04
Epoch 293/350
31/31  0s 4ms/step - loss: 6.5996e-04 - val_loss: 5.6850e-04
Epoch 294/350
31/31  0s 4ms/step - loss: 6.4314e-04 - val_loss: 5.7440e-04
Epoch 295/350
31/31  0s 4ms/step - loss: 6.5353e-04 - val_loss: 5.7100e-04
Epoch 296/350
31/31  0s 4ms/step - loss: 7.2492e-04 - val_loss: 5.7014e-04
Epoch 297/350
31/31  0s 5ms/step - loss: 7.4983e-04 - val_loss: 5.6864e-04
Epoch 298/350
31/31  0s 4ms/step - loss: 6.9108e-04 - val_loss: 5.6596e-04
Epoch 299/350
31/31  0s 4ms/step - loss: 6.5284e-04 - val_loss: 5.6752e-04
Epoch 300/350
31/31  0s 4ms/step - loss: 7.1888e-04 - val_loss: 5.7256e-04
```

```
Epoch 301/350
31/31  0s 4ms/step - loss: 7.5145e-04 - val_loss: 5.6867e-04
Epoch 302/350
31/31  0s 4ms/step - loss: 6.9928e-04 - val_loss: 5.6738e-04
Epoch 303/350
31/31  0s 4ms/step - loss: 7.2237e-04 - val_loss: 5.7326e-04
Epoch 304/350
31/31  0s 4ms/step - loss: 6.4127e-04 - val_loss: 5.7441e-04
Epoch 305/350
31/31  0s 4ms/step - loss: 6.5799e-04 - val_loss: 5.7343e-04
Epoch 306/350
31/31  0s 4ms/step - loss: 6.7468e-04 - val_loss: 5.7567e-04
Epoch 307/350
31/31  0s 4ms/step - loss: 6.7632e-04 - val_loss: 5.7237e-04
Epoch 308/350
31/31  0s 4ms/step - loss: 6.9157e-04 - val_loss: 5.7145e-04
Epoch 309/350
31/31  0s 4ms/step - loss: 6.7008e-04 - val_loss: 5.7113e-04
Epoch 310/350
31/31  0s 3ms/step - loss: 6.9951e-04 - val_loss: 5.6989e-04
Epoch 311/350
31/31  0s 4ms/step - loss: 6.4539e-04 - val_loss: 5.6870e-04
Epoch 312/350
31/31  0s 4ms/step - loss: 6.6928e-04 - val_loss: 5.7360e-04
Epoch 313/350
31/31  0s 4ms/step - loss: 6.7048e-04 - val_loss: 5.7327e-04
Epoch 314/350
31/31  0s 4ms/step - loss: 6.4485e-04 - val_loss: 5.7562e-04
Epoch 315/350
31/31  0s 4ms/step - loss: 6.9467e-04 - val_loss: 5.7438e-04
Epoch 316/350
31/31  0s 3ms/step - loss: 6.4320e-04 - val_loss: 5.7521e-04
Epoch 317/350
31/31  0s 4ms/step - loss: 6.7703e-04 - val_loss: 5.6928e-04
Epoch 318/350
31/31  0s 4ms/step - loss: 7.2475e-04 - val_loss: 5.7233e-04
Epoch 319/350
31/31  0s 4ms/step - loss: 6.8130e-04 - val_loss: 5.7276e-04
Epoch 320/350
31/31  0s 4ms/step - loss: 6.8497e-04 - val_loss: 5.6880e-04
Epoch 321/350
31/31  0s 4ms/step - loss: 6.8441e-04 - val_loss: 5.7140e-04
Epoch 322/350
31/31  0s 4ms/step - loss: 6.7678e-04 - val_loss: 5.7315e-04
Epoch 323/350
31/31  0s 4ms/step - loss: 6.4901e-04 - val_loss: 5.6909e-04
Epoch 324/350
31/31  0s 3ms/step - loss: 7.1097e-04 - val_loss: 5.7130e-04
Epoch 325/350
31/31  0s 4ms/step - loss: 7.2447e-04 - val_loss: 5.6986e-04
Epoch 326/350
31/31  0s 4ms/step - loss: 6.6603e-04 - val_loss: 5.6759e-04
Epoch 327/350
31/31  0s 4ms/step - loss: 7.2407e-04 - val_loss: 5.7266e-04
Epoch 328/350
31/31  0s 4ms/step - loss: 6.7521e-04 - val_loss: 5.6604e-04
Epoch 329/350
31/31  0s 4ms/step - loss: 6.7511e-04 - val_loss: 5.6804e-04
Epoch 330/350
31/31  0s 4ms/step - loss: 7.0720e-04 - val_loss: 5.7600e-04
```

```

Epoch 331/350
31/31  0s 4ms/step - loss: 6.2524e-04 - val_loss: 5.7016e-04
Epoch 332/350
31/31  0s 4ms/step - loss: 7.1132e-04 - val_loss: 5.7241e-04
Epoch 333/350
31/31  0s 3ms/step - loss: 6.7122e-04 - val_loss: 5.7555e-04
Epoch 334/350
31/31  0s 4ms/step - loss: 7.1109e-04 - val_loss: 5.6844e-04
Epoch 335/350
31/31  0s 4ms/step - loss: 6.3204e-04 - val_loss: 5.6854e-04
Epoch 336/350
31/31  0s 4ms/step - loss: 6.2831e-04 - val_loss: 5.7063e-04
Epoch 337/350
31/31  0s 4ms/step - loss: 6.6687e-04 - val_loss: 5.6860e-04
Epoch 338/350
31/31  0s 4ms/step - loss: 6.4531e-04 - val_loss: 5.6928e-04
Epoch 339/350
31/31  0s 4ms/step - loss: 6.7892e-04 - val_loss: 5.6799e-04
Epoch 340/350
31/31  0s 6ms/step - loss: 7.2527e-04 - val_loss: 5.6921e-04
Epoch 341/350
31/31  0s 5ms/step - loss: 6.7714e-04 - val_loss: 5.7220e-04
Epoch 342/350
31/31  0s 5ms/step - loss: 6.9821e-04 - val_loss: 5.6902e-04
Epoch 343/350
31/31  0s 4ms/step - loss: 7.7455e-04 - val_loss: 5.7205e-04
Epoch 344/350
31/31  0s 4ms/step - loss: 6.4468e-04 - val_loss: 5.7150e-04
Epoch 345/350
31/31  0s 4ms/step - loss: 6.8735e-04 - val_loss: 5.6835e-04
Epoch 346/350
31/31  0s 4ms/step - loss: 6.4893e-04 - val_loss: 5.6746e-04
Epoch 347/350
31/31  0s 4ms/step - loss: 6.7634e-04 - val_loss: 5.6932e-04
Epoch 348/350
31/31  0s 4ms/step - loss: 6.8348e-04 - val_loss: 5.6930e-04

```

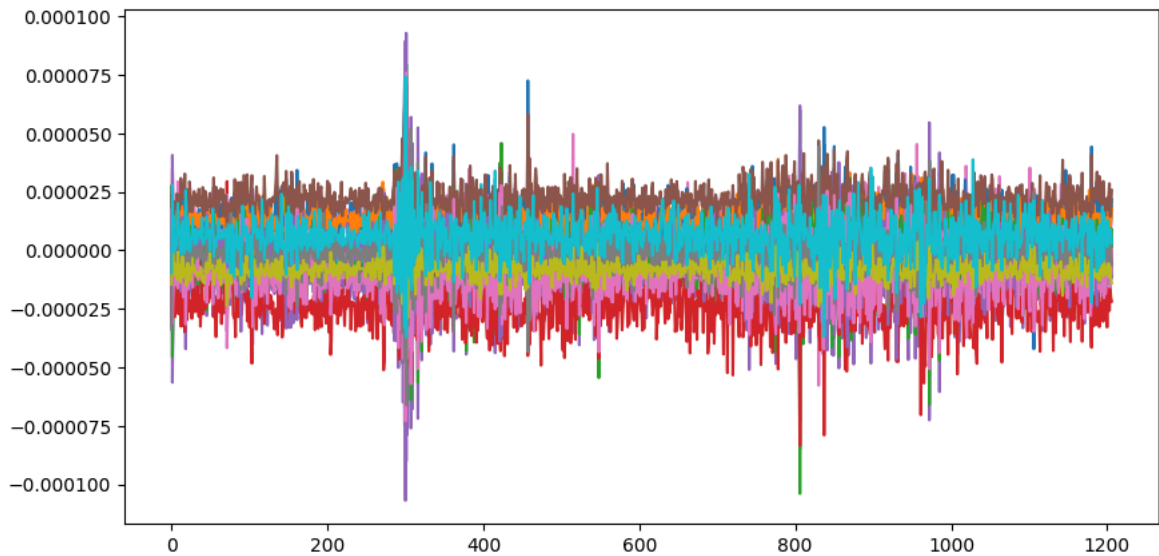
Out[8]: <keras.src.callbacks.history.History at 0x7f9295627e90>

By plotting the time series of the factors generated by the model, we can observe how the regularization term shrinks the dimensionality of the bottleneck. The model is forced to generate a number of latent factors with lower dimensionality than the data and that are relevant to explain its variation.

```

In [9]: factors = model.encoder(Y_train).numpy()
plt.figure(figsize=(10, 5))
plt.plot(factors)
plt.show()

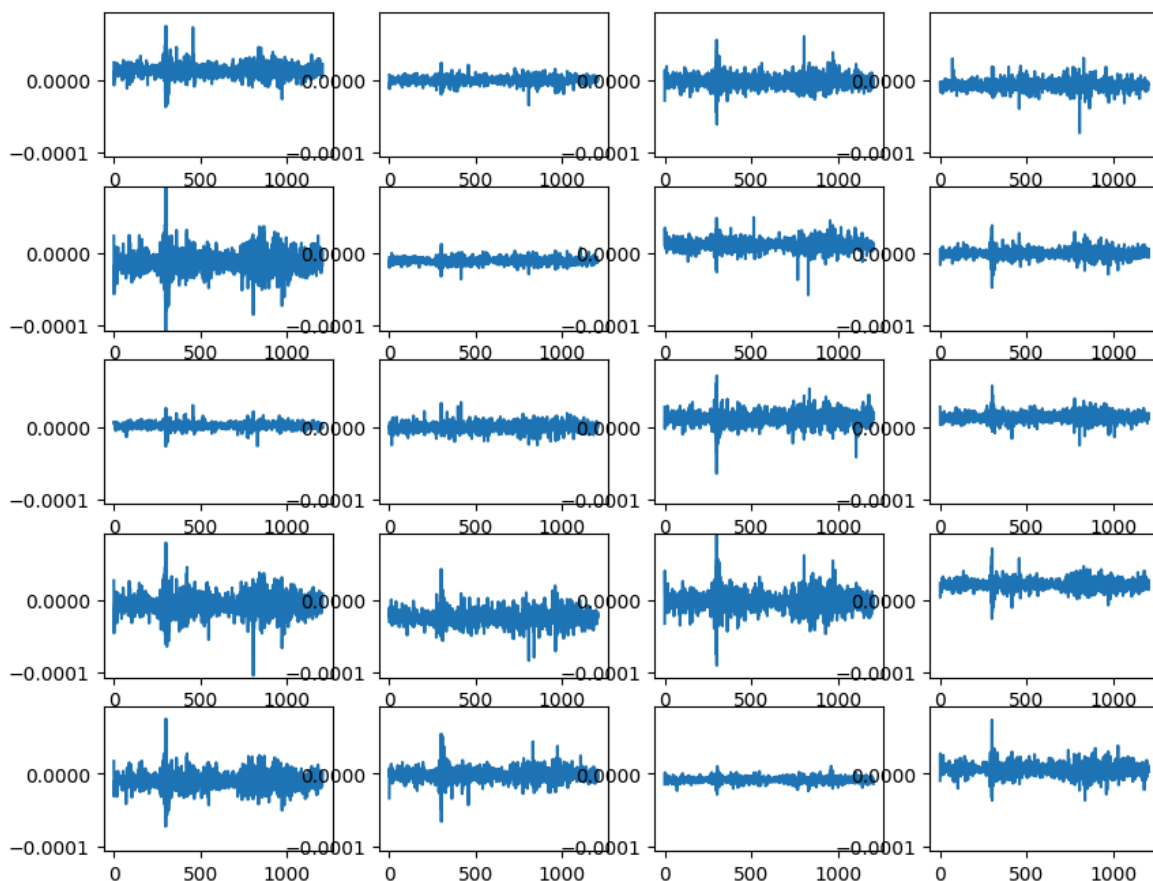
```



Below, we plot the time series of each factor. Observe how, out of 20 possible dimensions, the addition of the regularization term enables us to "drop" some dimensions. Notice the resemblance of this regularization technique with the L1-regularization of weights in MLPs. In that setup, we were interested in filtering a set of input features that are relevant for predictions. In the autoencoder setup, the activity regularization chooses a few factors that are relevant at explaining the variation in the data.

You can play with the parameter λ_1 above and observe its impact on the number of factors generated and the performance of the model. Notice also that the results depend on the random seed that is used to initialize the model's parameters while training.

```
In [10]: custom_ylim = [np.min(factors), np.max(factors)]
fig, axs = plt.subplots(5, 4, figsize=(10, 8))
for i in range(5):
    axs[i, 0].plot(factors[:, 0 + i * 4])
    axs[i, 0].set_ylim(custom_ylim)
    axs[i, 1].plot(factors[:, 1 + i * 4])
    axs[i, 1].set_ylim(custom_ylim)
    axs[i, 2].plot(factors[:, 2 + i * 4])
    axs[i, 2].set_ylim(custom_ylim)
    axs[i, 3].plot(factors[:, 3 + i * 4])
    axs[i, 3].set_ylim(custom_ylim)
```



In terms of reconstruction performance, the regularization parameter λ_1 is quite small, so the reconstruction errors from this model should be below those in Lesson 2 using an undercomplete autoencoder with three factors. This is a natural outcome because increasing the number of latent factors should increase the proportion of the variance explained by the model.

```
In [11]: Y_pred = model.predict(Y_train, verbose=0)
error = np.sum((Y_train - Y_pred) ** 2) / np.sum((Y_train - np.mean(Y_train)) ** 2)
print("Reconstruction error in training sample", error)
Y_pred = model.predict(Y_test, verbose=0)
error_oos = np.sum((Y_test - Y_pred) ** 2) / np.sum((Y_test - np.mean(Y_test)) ** 2)
print("Reconstruction error in test sample", error_oos)
```

Reconstruction error in training sample 0.9996295625952354

Reconstruction error in test sample 1.0017582451379592

3. PORTFOLIO CHOICE USING THE PREDICTIONS OF THE AUTOENCODER

Now we are going to use the trained autoencoder to choose an "optimal" portfolio of stocks. Based on the estimated exposures of each stock to the factors that determine the comovement in stock returns, we can design a diversified portfolio that maximizes the expected risk-weighted returns. Let's formally describe this.

Notice that the decoding layer of the autoencoder implies that we can model the returns of a stock i at time t as: $r_{it} = \alpha_i + \beta_{i1} f_{1t} + \beta_{i2} f_{2t} + \dots + \beta_{ik} f_{kt} + \epsilon_{it}$ where $f_{1t}, f_{2t}, \dots, f_{kt}$ are the

factors generated by the autoencoder, and $\alpha_i, \beta_{i1}, \beta_{i2}, \dots, \beta_{ik}$ are the trained parameters. ϵ_{it} are zero-mean error terms. The representation above is called a linear factor model.

If we stack the observations of returns of all the stocks $i=1, \dots, n$ at a period t in a column vector R_t , stock returns are given by: $R_t = \mathbf{a} + B F_t + \mathbf{e}_t$ where (bold faces are meant to differentiate vectors from scalars):

- $\mathbf{a}$ is an $n \times 1$ vector that includes all the trained α_i 's
- B is an $n \times k$ matrix that includes all the trained β_{ij} 's.
- F_t is a $k \times 1$ vector of realizations of the factors.
- $\mathbf{e}_t$ is an $n \times 1$ vector of idiosyncratic (stock-specific) errors ϵ_{it} 's

Assume that the variance of $\mathbf{e}_t$ is a diagonal $n \times n$ matrix Σ_e , while the variance of F_t is denoted by the $k \times k$ matrix Σ_f . These matrices capture the variation and covariation of the factors and idiosyncratic terms over time and that we can estimate, for instance, from our autoencoder model. Thus, the expectation and variance of the return vector R_t are given by $E(R_t) = \mu = \mathbf{a} + B E(F_t)$ $\text{Var}(R_t) = \Sigma = B \Sigma_f B^T + \Sigma_e$ The first component of the variance of returns is called the systematic component of the variance, which arises from the common latent factors that determine all the returns, depending on the stock-specific exposures to those factors included in B . The second component is the idiosyncratic component of the variance. This variation can be interpreted as noise, which is assumed to be stock-specific (uncorrelated across stocks).

Using the autoencoder, we can build our estimates for μ and Σ . For μ , we take the average of the reconstruction of each stock return over the training sample, $E(\hat{R}_t)$. For Σ , we first compute the variance covariance matrix of the reconstructions, $\Sigma_f = \text{Var}(\hat{R}_t)$, and then we add the idiosyncratic component from the variance-covariance matrix of the reconstruction errors, $\Sigma_e = \text{diag}(\text{Var}(\mathbf{e}_t))$. Following our assumption above, we impose a diagonal Σ_e . Although the matrix $\text{Var}(\mathbf{e}_t)$ may not be diagonal, we treat the remaining off-diagonal variation just as "noise" that is irrelevant for generalization.

```
In [12]: Y_pred = model.predict(Y_train, verbose=0)
mu = np.mean(Y_pred, axis=0)
sigma_f = np.cov(Y_pred, rowvar=False)
sigma_e = np.cov(Y_train - Y_pred, rowvar=False)
sigma_d = np.diag(np.diag(sigma_e))
sigma = sigma_f + sigma_d
```

Let's now consider building a portfolio with the modelization approach that we have derived above. A portfolio is defined as a vector $w = (w_1, \dots, w_n)^T$ where each w_i represents the proportion of the investor's wealth that we invest in an asset i . We must impose the constraint $\sum_i w_i = 1$, meaning that the investor has all of its wealth invested in the n assets. This does not rule out the possibility of short-selling, $w_i < 0$

0\$, so that the portfolio can take long positions on some assets and short positions on others.

Given a choice of portfolio w on the n assets, the return of the portfolio in period t is given by $R_{w,t} = w^T R_{t}$. Hence, the expected return and variance of this portfolio can be denoted by the following scalars: $E(R_{w,t}) = w^T \mu$ $Var(R_{w,t}) = w^T \Sigma w$

Let's define the Sharpe Ratio, SR , of a portfolio as its expected return per unit of risk, i.e.: $SR_w = \frac{E(R_{w,t})}{\sqrt{Var(R_{w,t})}} = \frac{w^T \mu}{\sqrt{w^T \Sigma w}}$

Note: Formally, to compute a Sharpe Ratio we should subtract the risk-free rate from the numerator, but we are going to abstract from it. You can read the [Investopedia entry](#) as further reference.

We are going to use the Sharpe Ratio as the performance metric that determines whether a portfolio is better than other. Thus, our objective is to find the portfolio that generates the maximum Sharpe Ratio given our estimates of expected returns and variance covariance matrix of returns.

For this, we set up an optimization problem, where the objective function is the Sharpe Ratio, given vectors of weights, expected returns, and the variance-covariance matrix. We put a minus term in front as we are going to define the optimization problem as a minimization objective.

```
In [13]: def sharpe_ratio(weights, mean, var):  
         return -np.dot(weights, mean) / np.sqrt(np.dot(np.dot(weights, var), weights
```

We then define a function where the optimization problem is performed, including the constraint that $\sum_i w_i = 1$. If we were interested in imposing a long-only portfolio (short-selling constraint) then we should change the "bound" tuple to take the value $(0,1)$, instead of $(None, None)$.

```
In [14]: import scipy.optimize as sco
```

```
In [15]: def max_sharpe(mean, var):  
         args = (mean, var)  
         num_assets = len(mean)  
         args = (mean, var)  
         constraints = {"type": "eq", "fun": lambda x: np.sum(x) - 1}  
         bound = (None, None)  
         bounds = tuple(bound for asset in range(num_assets))  
         result = sco.minimize(  
             sharpe_ratio,  
             num_assets  
             * [  
                 1.0 / num_assets,  
             ],  
             args=args,  
             method="SLSQP",  
             bounds=bounds,
```

```

        constraints=constraints,
        tol=1e-12,
    )
    return result

```

Given, the estimates of μ and Σ from the autoencoder, we get the following portfolio weights for the 20 stocks. We also verify that the constraint is roughly satisfied.

```

In [16]: opt = max_sharpe(mu, sigma)
         opt.x

```

```

Out[16]: array([ 0.13625731,  0.09874778,  0.0523024 ,  0.03819203,  0.06961968,
                 0.00350657,  0.05610333,  0.05169225, -0.00699324, -0.01166656,
                 0.07362228,  0.06228513,  0.07470726,  0.03380913,  0.04850011,
                 0.03878507,  0.02906145,  0.07333414,  0.05462749,  0.02350638])

```

```

In [17]: np.sum(opt.x)

```

```

Out[17]: 1.0

```

Actually, from portfolio choice theory, we already know the analytical expression for the portfolio with maximum Sharpe Ratio in the absence of short-selling constraints: $w^* = \frac{\Sigma^{-1}\mu}{\mathbf{1}^T \Sigma^{-1}\mu}$ where $\mathbf{1}$ is an $n \times 1$ vector of ones. See this [link](#) for a proof of this result. Below, we show that the results from the optimization problem are roughly identical (reduce the tolerance of the optimization problem to obtain closer results).

```

In [18]: # Analytic solution (only when short positions are allowed)
         weights = np.dot(np.linalg.inv(sigma), mu)
         weights = weights / np.sum(weights)
         weights

```

```

Out[18]: array([ 0.13625741,  0.09874773,  0.05230228,  0.03819205,  0.06961961,
                 0.00350667,  0.05610339,  0.05169231, -0.00699334, -0.01166649,
                 0.07362227,  0.06228522,  0.07470719,  0.03380929,  0.04850003,
                 0.03878504,  0.02906152,  0.0733341 ,  0.05462746,  0.02350625])

```

Now, let's look at the performance of the portfolio in the training sample and in the test sample. Note that the results may vary depending on the seed used while training. You can train the model several times to have a general view on the out-of-sample performance of the strategy.

```

In [19]: mu_true_test = np.mean(Y_test, axis=0)
         sigma_true_test = np.cov(Y_test, rowvar=False)
         print(
             "Sharpe ratio in training (estimated): ",
             -np.sqrt(252) * sharpe_ratio(opt.x, mu, sigma),
         )
         print(
             "Sharpe ratio in test (actual): ",
             -np.sqrt(252) * sharpe_ratio(opt.x, mu_true_test, sigma_true_test),
         )

```

```

Sharpe ratio in training (estimated):  3.3954331835003013
Sharpe ratio in test (actual):  2.07543332497095

```

Does the portfolio obtained from the autoencoder perform better than a "naive" portfolio that simply uses the mean returns and variance covariance matrix of returns over the training sample to construct the portfolio? Below, we compare the autoencoder model with that benchmark.

```
In [20]: mu_true = np.mean(Y_train, axis=0)
sigma_true = np.cov(Y_train, rowvar=False)
opt = max_sharpe(mu_true, sigma_true)
print(
    "Sharpe ratio in training: ",
    -np.sqrt(252) * sharpe_ratio(opt.x, mu_true, sigma_true),
)
print(
    "Sharpe ratio in test (actual): ",
    -np.sqrt(252) * sharpe_ratio(opt.x, mu_true_test, sigma_true_test),
)
```

```
Sharpe ratio in training:  1.9655875931884363
Sharpe ratio in test (actual):  1.6941130374559856
```

Below, we show the results from choosing the optimal portfolio using estimates from PCA, using as many principal components as those obtained from the autoencoder model. Since this method does not produce estimates of the average returns, we rely on using the principal components (which are calculated from the centered or demeaned data) to calculate the average returns over the training sample as estimates for the expected returns, μ .

Notice that the standard PCA method does not use a validation sample to avoid overfitting, which naturally leads to different results than an autoencoder approach. However, the results are independent from the choice of random seed.

```
In [21]: from sklearn.decomposition import PCA

pca = PCA(n_components=17)
pca.fit(Y_train)
pca_factors = pca.transform(Y_train)
Y_pca_pred = np.dot(pca_factors, pca.components_)
sigma_f = np.cov(Y_pca_pred, rowvar=False)
sigma_e = np.cov(Y_train - Y_pca_pred, rowvar=False)
sigma_d = np.diag(np.diag(sigma_e))
sigma_pca = sigma_f + sigma_d
```

Below we compute the Sharpe ratios of the portfolios built using PCA. The estimated Sharpe ratio from the PCA is 2.02 (in training), but in the test sample, it drops to -0.31. Notice that the optimal portfolio that arises from PCA will give greater weight to stocks with higher average returns over the training sample. Hence, the Sharpe ratio turns negative because those stocks that performed well in the training sample seem to have performed badly in the test sample.

```
In [22]: opt = max_sharpe(mu_true, sigma_pca)
print(opt.x)
print(
    "Sharpe ratio in training (estimated): ",
    -np.sqrt(252) * sharpe_ratio(opt.x, mu_true, sigma_pca),
)
```

```
)
print(
    "Sharpe ratio in test (actual): ",
    -np.sqrt(252) * sharpe_ratio(opt.x, mu_true_test, sigma_true_test),
)
```

```
[ 1.45251094  1.02761215  2.5769373  -1.79584562  0.87488948  0.49026512
 1.87165622 -0.07636212 -0.06377933 -1.14006306  0.21441525  3.11683583
 2.1610222  -2.3176693  -1.57076272 -0.84432432 -0.21190979 -3.12019528
-0.83857693 -0.80665601]
```

```
Sharpe ratio in training (estimated): 2.016511478146714
```

```
Sharpe ratio in test (actual): 2.0072948864942446
```

5. CONCLUSION

Although the autoencoder may not generate extremely satisfactory results, its performance may improve upon other more naïve estimation procedures. The results are also largely dependent on the initialization of the autoencoder's weights. In the next module, we'll deal with ensemble methods that, at least partly, deal with this annoying variability.

Notice also that the train and test samples display very different market environments. It is quite challenging to find machine learning algorithms that adapt well to such changes in the environment.

Lastly, as you have probably grasped in the previous modules of this course, obtaining a good deep learning model for finance applications requires a great deal of tweaking and understanding of how the model is a suitable tool for the kind of data that we have at hand.

With this notebook, we finalize the descriptions of autoencoders and their application to financial time series.

In Module 7, we will study further advanced topics in Deep Learning for Finance.

See you there!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.